Artifact Appendix

Chasing Cookies to the Source: In-browser Data Flow Backtracking for Cookie Compliance Triage

A. Description & Requirements

CookieTrace traces cookie writes back to contributing literals and scripts through in-browser data-flow backtracking. It extends Arcanum and Chromium 108.0.5359.71 with runtime maps, selective data-flow construction, forward-call tracking, and asynchronous stack stitching. The artifact contains the browser modifications, execution scripts, analysis scripts, and experimental records.

How to access. Repository: https://github.com/windY1Y/CookieTrace. Start with ae/README.md. The experiment-to-script mapping, commands, and output descriptions are in ae/REPRODUCING.md. The evaluation layout preserves the repository's measure/, data_processing/, and statistics/ directories and adds the portable ae/ entry point.

Hardware dependencies. An x86-64 Linux desktop or laptop is recommended, with 8 CPU cores and 16 GB RAM and one browser worker at a time. No GPU or special hardware is required. Provide storage for the browser runtime, Docker image, experimental records, and outputs; the chrome executable alone is approximately 1.65 GB.

Software dependencies. Bash, Python >=3.9, and a local x86-64 Linux Docker Engine are used for portable execution. The Dockerfile installs Debian 12, Xvfb, and browser libraries. Use the modified Chromium 108.0.5359.71 runtime bundle. Analysis uses Python scientific/plotting packages and Node.js for the JavaScript analyses. Internet access is needed for setup and live sites; local examples and saved-record analyses do not require external websites after setup. The paper's live measurements used German network egress.

Benchmarks. The benchmarks cover consent cookies and unnecessary cookies collected from websites. The effectiveness evaluation uses an original sample set and a deduplicated subset. The statistics/ directory contains their effectiveness CSVs and processed characterization results. Source/path records provide the inputs for source-characterization analysis; instrumented and vanilla browser timing records provide the inputs for performance analysis.

B. Artifact Installation & Configuration

Place the matching browser runtime in ae/browser/sys/, so its executable is ae/browser/sys/chrome. From the repository root, run:

```
chmod u+x ae/browser/sys/chrome
bash ae/run.sh --build
bash ae/run.sh --check
bash ae/run.sh --demo
```

The build action constructs the Docker runtime; it does not compile Chromium. The checker inspects the browser bundle and runtime dependencies. Each browser run uses Xvfb, a fresh profile, and a unique output directory under ae/results/. A bind mount maps the engine's fixed internal output directory to the current run's raw/ directory. --browser-dir, --output-dir, and --proxy configure the evaluator's paths and network.

C. Experiment Workflow

Run the local example (E1), compute accuracy summaries (E2), analyze recorded runtime overhead (E3), and aggregate source characteristics (E4). measure/single_site.sh is the original per-website browser entry. ae/run.sh provides portable browser execution using --url and --cookie-name. Statistical scripts consume the records described in ae/REPRODUCING.md; they are separate from browser startup.

D. Major Claims

(C1) Source accuracy. CookieTrace identifies the source literals contributing to cookie writes. E2 computes source precision and recall for the original and deduplicated evaluation sets, corresponding to the CookieTrace source-accuracy entries in Tables II-III.

(C2) Runtime overhead. CookieTrace runtime cost is evaluated through instrumented and vanilla browser load times and analysis-stage timings. E3 aggregates the measurement records for the performance analysis in Figure 4.

(C3) Source characteristics. CookieTrace exposes forward-call dependence, propagation structure, source-script concentration, script categories, and collaboration. E4 processes source and path records for the analyses in Figures 2-3 and 5-6 and Tables IV-V.

E. Evaluation

Experiment (E1): Browser execution. [Preparation] Place the browser bundle in ae/browser/sys/, build the Docker runtime, and run the environment check. [Execution] Run bash ae/run.sh --demo for the local example. For a live website, invoke the launcher with --url URL and --cookie-name NAME; use --timeout to configure the time limit. The original measure/single_site.sh also provides a per-website execution entry. [Output inspection] Open the generated run directory under ae/results/. Check the status record, sources.json, paths.json, raw trace files, and browser log. Confirm that the output corresponds to the requested cookie and contains the expected source literals and completed path metadata.

Experiment (E2): Effectiveness (C1). [Preparation] Select statistics/effectiveness_original.csv and statistics/effectiveness_deduplicated.csv. [Execution] From the repository root, run python3 ae/check_effectiveness.py . The checker reads the per-cookie counts, excludes the aggregate Total row from the sample calculations, and verifies that the aggregate agrees with the individual records. [Output inspection] Inspect the reported TP, FP, FN, precision, and recall for each dataset. Micro precision aggregates source counts; macro precision averages per-cookie ratios. Use the corresponding metric to interpret the CookieTrace source-accuracy entries in Tables II-III. This workflow recomputes metrics from the supplied count records.

Experiment (E3): Performance (C2). [Preparation] Select the instrumented load-time records, corresponding path JSON timestamps, and vanilla-browser timing records. [Execution] Run data_processing/summarize_overhead.py with --json_dir for the measurement root, --mode for the cookie group, --clean_file for vanilla timing records, and --output for the result JSON. The script joins records by domain and instance, applies its load-time filters, and produces per-site stage intervals, load times, and slowdown ratios. Pass the resulting JSON through standard input to data_processing/compute_avg_time.py to aggregate its numeric fields. [Output inspection] Inspect the per-site records, included samples, and aggregate timings for the Figure 4 analysis. Use recorded page-load measurements for this workflow; the launcher elapsed_seconds field describes its own execution duration.

Experiment (E4): Characterization (C3). [Preparation] Select paired source/path records with their original domain and instance identifiers. [Execution] Run ae/analysis/char_new.py --dir RAW_ROOT --out SUMMARY_DIR to produce per-case and global summaries. In ae/analysis/, draw_cdf_forward_call_num.py reads these summaries to plot forward-call counts for Figure 2; draw_forward_call_dependent.py plots forward-dependent source fractions for Figure 3; draw_kde.py compares propagation and stack depths for Figure 5. Supply the summary directory through --input and select an output file through --out. Keep preprocessing choices explicit; --method none retains untrimmed observations.

collect_source_scripts.py aggregates normalized source-script URLs for the Table IV analysis. aggregate_by_category.py aggregates classified script entries for the Figure 6 category analysis. collboration.py summarizes single-/multi-script and first-/third-party combinations using total_participated_scripts for the collaboration analysis associated with Table V. The category aggregator consumes classified entries, while the source collector emits URL/count/example records. Additional domain and cookie-decoding analyses are in data_processing/characterization/.

[Output inspection] Inspect the per-case and global JSON summaries, source-script lists, category aggregates, collaboration summaries, and generated plots. Preserve website/cookie identifiers and use consistent participation definitions, URL normalization, and deduplication policies across analyses. The detailed commands and script inputs are described in ae/REPRODUCING.md. Keep outputs from fresh live-site runs separate from analyses of the supplied records.